# FIXO — Architecture & Workflow Specification (Pre-Implementation)

Oct 1, 2026 · @Prince Kumar

## Overview

This is the complete pre-implementation design for FIXO V1 in Lucknow, structured as sections A–Z from your brief. No code is written until you approve it.

Seven design decisions drive everything below:

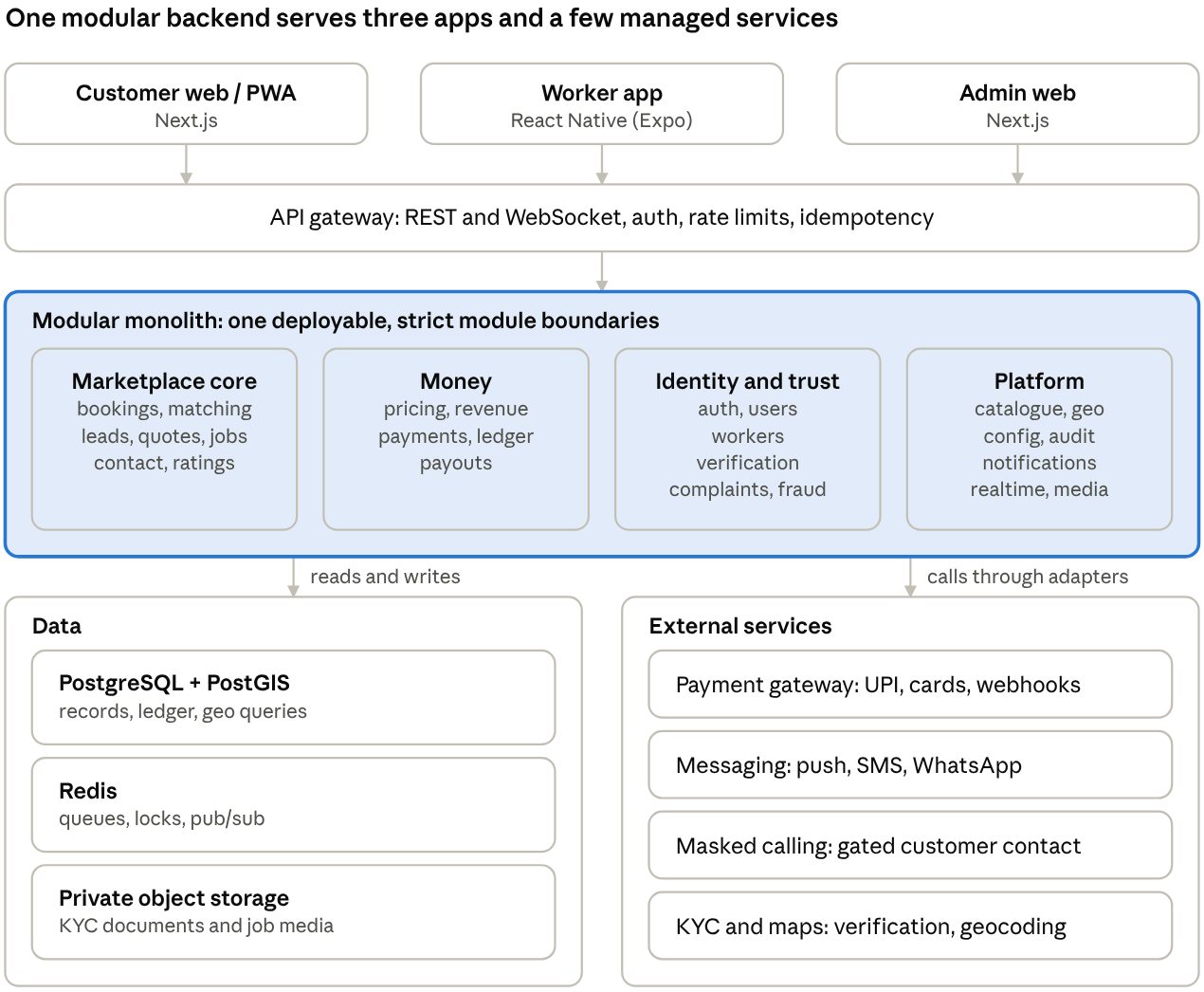
1. **Modular monolith first.** One deployable backend with strict module boundaries. Split into services only when load proves it necessary.
2. **Separate state machines.** Booking, lead, quote, job, payment and settlement each have their own status. They are never merged into one field.
3. **Everything business-related is configuration.** Fees, commission, radius, lead caps, price ranges and cancellation rules live in versioned tables. Every booking stores a snapshot of the rules that applied when it was created.
4. **Money is append-only.** Financial history comes from an immutable ledger, never from current booking fields.
5. **Leads are capped and contact is gated.** A lead goes to a ranked set of workers; once the cap is reached it closes. Customer phone numbers are revealed only after lead access is granted, through a masked channel.
6. **The server owns the truth** for price, commission, earnings, role, KYC status and payment status. The client only sends intent.
7. **Value beyond the first call.** Quotes, payment protection, warranty history, receipts and disputes are what keep customers and workers on the platform, not hidden phone numbers alone.

Anything your brief leaves ambiguous is marked **OPEN BUSINESS DECISION (OBD-n)** where it arises and collected, with options and a recommendation, in the section *Open business decisions* near the end.

**Assumptions used here (change any of them):** India only, INR only, Hindi + English UI, UPI and cash as the main payment methods, and Lucknow as the single launch city.

# A. System architecture

FIXO runs as one modular monolith backed by PostgreSQL, with Redis for queues, locks and realtime fan-out. Modules talk through internal interfaces and domain events, so any module can later become a service without a rewrite.




system architecture · 3 apps, 1 backend, data and external services

Apps talk only to the gateway. Modules share one database and queue, and every outside service is reached through an adapter.

**Recommended stack (a proposal, not a constraint):**

| Layer | Choice | Why |
|---|---|---|
| Language | TypeScript end to end | One language across API, web and mobile |
| Backend | NestJS (modules, guards, validation) | Enforces module boundaries and RBAC |
| Database | PostgreSQL + PostGIS | Transactions for money, geo queries for matching |
| Cache, queues, locks | Redis + BullMQ | Lead expiry timers, retries, distributed locks |
| Realtime | WebSocket gateway (Socket.IO) with Redis adapter | Lead pushes, job status; scales across instances |
| Customer client | Next.js web/PWA first; native app later | Lowest friction to first booking |
| Worker client | React Native (Expo), Android first | Push alerts and fast lead response matter most |
| Admin | Next.js web app | Desktop-first, table heavy |
| Files | S3-compatible private bucket, signed URLs | KYC and job media never public |
| Payments | Razorpay (UPI, cards) plus a cash path | Common choice in India |
| Messaging | Push (FCM), an Indian SMS/OTP provider, WhatsApp Business API | Multi-channel with fallback |
| Masked calling | Cloud telephony provider with number masking | Gated contact without sharing numbers |
| Maps | Google Maps or an OSM-based provider | Geocoding, distance, area polygons |

**Scaling beyond Lucknow without a rewrite:**

- Every operational row carries `city_id`. Nothing in code mentions a city name.
- Geography is data: Country → State → City → Area → Service Zone, each with a polygon and an active flag.

- Configuration resolves by scope, most specific wins: **global → city → zone → category → worker plan**. Adding a city is a data change plus an admin activation, not a deployment.
- Hot read paths (service catalogue, area lists, pricing rules) are cached in Redis and invalidated on admin change.

**Cross-cutting rules in the architecture:**

1. Every write that moves money or changes a lead or booking state runs inside a database transaction.
2. Every such endpoint accepts an idempotency key.
3. Every state change appends a row to a status-history table and emits a domain event.
4. Notifications, matching and payouts run as background jobs, never inside the request.

## B. Customer flow

A customer reaches a confirmed booking in about six taps, and is asked for a profile only when it is needed.

1. **Open app, browse.** No login required to see categories, search and suggested price ranges.
2. **Pick category, then problem.** Structured selection from the admin-managed catalogue (Category → Subcategory → Problem). The problem screen shows the suggested price range, estimated duration and whether emergency service is available.
3. **Optional photo or video.** Uploaded to private storage; skippable.
4. **Location.** Current location, saved address or manual pin. The server resolves coordinates to city, area and service zone. Outside active zones, the customer sees a "not in your area yet" screen and can join a waitlist.
5. **Time.** ASAP, Today, Tomorrow or a date, plus an admin-configured slot (for example 09:00–12:00).
6. **Login at the last step.** Mobile number + OTP, only at submit. Name is the only required profile field.
7. **Review and submit.** Shows the problem, address, time, suggested price range and booking fee (if any). The booking is created as REQUESTED and a duplicate-booking check runs first.
8. **Waiting screen.** Live status: workers notified → worker interested → quotes received. Customer never receives raw calls from unlimited workers.

9. **Compare and choose.** Up to the configured cap of workers appear as cards (name, photo, rating, jobs done, verification badges, distance, quote once submitted). The customer accepts a quote, rejects it, asks for a new one or cancels.
10. **Track the job.** On the way → arrived → in progress → completed, with push updates.
11. **Confirm completion, pay, rate.** Confirm or report an issue; pay online or mark cash; rate on five dimensions.
12. **After the job.** Receipt, service history, warranty note and a **Book again** button (same worker or any worker).

**Branches the customer can hit:**

- No workers available → the booking stays open, the radius widens in configured steps, then the customer is told honestly and offered a scheduled slot or cancellation.
- Customer cancels at any step → follows the cancellation matrix in section H.
- Quote too high → reject, request revision, or ask FIXO support to help.

## C. Worker flow

A worker should be able to respond to a lead in under ten seconds.

**Onboarding (one-time):**

1. Mobile OTP verification.
2. Basic profile: name, photo, city.
3. Areas covered (from the admin-managed list), preferred areas, maximum travel distance.
4. Services and skills, experience, pricing preferences.
5. KYC: identity document, selfie, bank/UPI details. Stored privately, never shown to customers.
6. Agreement acceptance (stored with version and timestamp).
7. Admin review: KYC decision and, separately, skill verification. Status moves REGISTERED → KYC_PENDING → UNDER_REVIEW → VERIFIED (or REJECTED).

Only VERIFIED workers who are online receive normal leads.

**Daily loop:**

1. **Go online** and set availability (hours, days, maximum active jobs, emergency on/off).
2. **Receive a lead** by push (and realtime if the app is open): problem, area, approximate distance, time, photo, suggested price, lead fee.

3. **Respond** within the lead window: **Interested**, **Ignore** or **Not available**. A single tap on Interested applies the configured lead-access rule (fee, credit, plan or none).
4. **If the lead is still open**, the worker gets contact access: masked call or in-app chat, and the exact address only after confirmation (see OBD-6).
5. **Call the customer** and agree scope.
6. **Submit a quote** in the app: visit, labour, material estimate, discount, total, notes.
7. **Wait for the customer decision.** A confirmed job appears with full address, agreed price and time.
8. **Execute:** On the way → Arrived → Work started → Work completed (with final amount, summary, optional photo).
9. **Additional work** goes through a new quote, never verbally.
10. **Collect payment** (cash or online) and see the earnings breakdown: gross → fees → FIXO commission → net.
11. **Rate the customer** and close the job.

**Branches:**

- Lead closes before the worker responds → shown as "lead filled", no fee charged.
- Worker goes offline mid-lead → in-flight accepted leads stay valid; new ones stop.
- Worker is suspended mid-job → follows edge case 16 in section V.

## D. Admin flow

Admin work falls into five recurring loops. Each loop has a queue, a decision screen and an audit trail.

| Loop | Trigger | Admin action | System effect |
|---|---|---|---|
| Worker verification | Worker submits onboarding | Review KYC; separately mark skill verified per category; approve, reject with reason, or request resubmission | Worker status changes; notification sent; audit row written |
| Catalogue and area setup | Business change or new city | Create/edit category, subcategory, problem, price guideline, zone, slot window | New version created; cache invalidated; old bookings keep their snapshot |

| Loop | Trigger | Admin action | System effect |
|---|---|---|---|
| Marketplace tuning | Metrics or complaints | Change lead cap, radius, fees, commission, ranking weights for a scope | New versioned rule with effective-from date; never overwrites history |
| Live operations | Stuck lead, no-worker booking, price anomaly | Reassign, re-open or close leads; contact worker or customer; flag or warn | Status history and audit rows written |
| Disputes and finance | Complaint, refund, payout run | Review evidence, decide, issue refund or adjustment, approve payouts | Ledger entries appended; parties notified |

**Daily admin routine the UI is built around:**

1. Open the overview: bookings by funnel stage, leads with no responders, jobs at risk, open complaints.
2. Clear the verification queue.
3. Work the exceptions queue (no-worker bookings, price anomalies, expiring KYC, failed payments).
4. Handle disputes by oldest first.
5. Review the settlement and payout batch before approval.

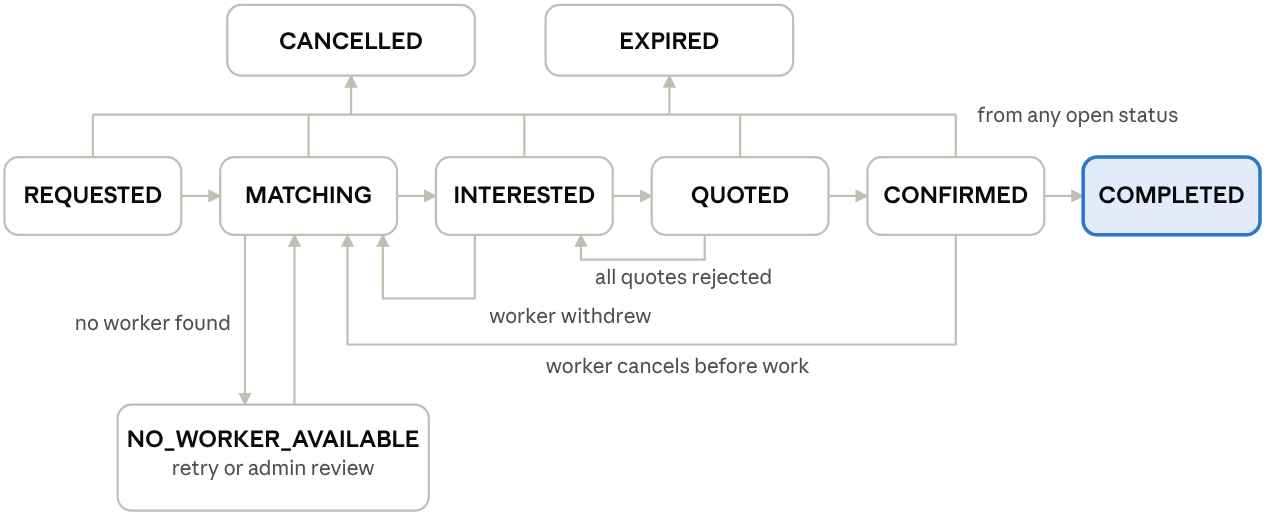
Admin roles are separate (super admin, operations, verification, finance, support), so a support agent cannot change commission and an operations user cannot approve a payout.

## E. Booking lifecycle

**Design correction to the brief's status list.** Your section 47 lists one long chain (REQUESTED … PAID → RATED). Putting it in one field would mix five different things, which you also warn against in section 29. FIXO therefore keeps **separate state machines** (booking, lead, quote, job, payment) and builds the customer-visible journey and the funnel analytics from their combined history. The long chain in your brief is preserved as a derived timeline, not a stored status.

The booking is the customer's request. It answers one question: *where is this request in finding and confirming a worker?*

**Bookings move toward COMPLETED and loop back to MATCHING when workers drop out**




booking status flow · 6 main states, 3 loops back, 2 exits

The booking moves left to right. Three loops return it to an earlier status: a worker withdraws, all quotes are rejected, or a confirmed worker cancels before work starts.

| Status | Meaning | Entered when | Can move to |
|---|---|---|---|
| REQUESTED | Validated and stored; duplicate check passed | Customer submits | MATCHING, CANCELLED |
| MATCHING | Ranked worker list built; leads released in waves | Matching job starts | INTERESTED, NO_WORKER_AVAILABLE, CANCELLED, EXPIRED |
| INTERESTED | At least one worker took lead access | First worker response | QUOTED, MATCHING (worker withdrew), CANCELLED, EXPIRED |

| Status | Meaning | Entered when | Can move to |
|---|---|---|---|
| QUOTED | At least one quote awaits the customer | First quote submitted | CONFIRMED, INTERESTED (all quotes rejected), CANCELLED, EXPIRED |
| CONFIRMED | Customer accepted a quote; a job exists | Quote accepted | COMPLETED, MATCHING (worker cancelled before work), CANCELLED |
| COMPLETED | Job completed and confirmed or auto-confirmed | Job reaches COMPLETED | Terminal (disputes live separately) |
| NO_WORKER_AVAILABLE | Radius widened and retried; still nobody | Matching exhausted | MATCHING (admin or retry), CANCELLED, EXPIRED |
| CANCELLED | Cancelled by customer, worker (final), or admin | Cancellation recorded with reason and actor | Terminal |
| EXPIRED | No decision within configured time | Timer fires | Terminal |

**Rules:**

1. Only the booking service changes `booking.status`. Every change appends a row to `booking_status_history` (from, to, actor, reason, timestamp).
2. A booking has at most one active job. If a worker cancels before work starts, that job is closed as CANCELLED_BY_WORKER and the booking returns to MATCHING with priority.

3. "Paid" and "Rated" are not booking statuses. They are facts from the payment and rating records.
4. The booking stores a **snapshot** of the pricing, fee and commission rule versions that applied at creation.
5. A repeat request (**Book again**) creates a new booking that references the old one; the old booking is never reopened.

**Derived customer timeline** (built from status histories, used for the app and funnel analytics): Requested → Matched → Interested → Contact unlocked → Called → Quoted → Confirmed → On the way → Arrived → In progress → Completed → Paid → Rated.

## F. Lead lifecycle

A **lead** is one booking offered to one worker. A booking therefore has many leads, but only a configured number of them can reach contact access. This is how a customer avoids ten random calls.

| Lead status | Meaning | Next states |
|---|---|---|
| OFFERED | Lead created for a ranked, eligible worker; notification queued | NOTIFIED, WITHDRAWN |
| NOTIFIED | Push/SMS delivered or attempted | VIEWED, EXPIRED, CLOSED_FILLED |
| VIEWED | Worker opened it | INTERESTED, DECLINED, UNAVAILABLE, EXPIRED, CLOSED_FILLED |
| INTERESTED | Worker tapped Interested; access attempt in progress | ACCESS_GRANTED, CLOSED_FILLED |
| ACCESS_GRANTED | Fee/credit/plan rule satisfied; contact unlocked | QUOTED, WITHDRAWN, LOST (customer picked another) |
| QUOTED | Worker submitted a quote | WON, LOST, WITHDRAWN |
| WON / LOST | Customer accepted this quote / chose another or cancelled | Terminal |

| Lead status | Meaning | Next states |
|---|---|---|
| DECLINED / UNAVAILABLE | Worker tapped Ignore / Not Available | Terminal (feeds response-history ranking) |
| EXPIRED | Lead window ended with no response | Terminal |
| CLOSED_FILLED | Cap reached before this worker got access; no charge | Terminal |
| WITHDRAWN | Worker or admin pulled out, or booking cancelled | Terminal |

**Distribution in waves (all values admin-configurable per scope):**

1. Matching ranks eligible workers (section Q).
2. Wave 1 releases to the top K workers (default proposal: 5).
3. If fewer than the cap have taken access after the wave window (default proposal: a few minutes), wave 2 releases to the next K, and so on, up to the maximum notified.
4. Emergency bookings skip waves and notify all eligible workers in the emergency radius at once.
5. Each lead expires after the lead window; the booking moves to NO_WORKER_AVAILABLE if all waves end with nobody.

**Cap enforcement (the critical concurrency rule):**

- Each booking has a `lead_pool` row holding the cap, expiry and a counter of granted accesses.
- Granting access runs in one transaction that locks the pool row, re-checks `granted < cap`, charges the lead fee through the ledger, writes the contact-access record and increments the counter.
- If two workers tap at the same moment, the lock serialises them. The second either gets the last slot or sees CLOSED_FILLED with no charge.
- The request carries an idempotency key, so a retry cannot charge twice or take two slots.

**Contact access** is a separate record (`contact_access`) with worker, booking, channel (masked call, chat, direct number), granted-at and expiry. Revoking access (worker suspended, booking cancelled) is a status change on that record, not a delete.

The lead fee timing and refund-if-unreachable policy are **OBD-2**; the cap and selection method are **OBD-3**.

## G. Quote lifecycle

A quote is an immutable offer from one worker for one booking. Changing a price creates a new revision; old revisions are never edited. This gives disputes a clean record of what was agreed.

**Quote fields:** visit charge, labour charge, material estimate (with note on what is included), discount, notes, validity period. The **total is computed by the server** from these fields, never accepted from the client.

| Quote status | Meaning | Next states |
|---|---|---|
| SUBMITTED | Worker sent it; validated against the pricing guideline | ACCEPTED, REJECTED, REVISION_REQUESTED, SUPERSEDED, EXPIRED, WITHDRAWN |
| REVISION_REQUESTED | Customer asked for a changed price or scope | SUPERSEDED (worker sends new revision), EXPIRED |
| SUPERSEDED | A newer revision replaced it | Terminal |
| ACCEPTED | Customer accepted; job is created | Terminal |
| REJECTED | Customer declined | Terminal |
| EXPIRED | Validity period ended | Terminal |
| WITHDRAWN | Worker withdrew before acceptance | Terminal |

**Quote types:**

- **INITIAL:** attached to a booking, accepted once to create the job.
- **ADDITIONAL:** attached to an active job when extra work or material is needed. Work on the extra scope does not start until the customer accepts. Rejection means the job continues on the original scope or is cancelled under the matrix in section H.

**Rules:**

1. A worker can have one live quote per booking. A new revision supersedes the previous one atomically.
2. Accepting a quote locks the others on that booking (they become LOST at the lead level) in one transaction.
3. Each line is checked against the effective pricing guideline for the problem, zone and scope. Out-of-range amounts are allowed but flagged (see section P); repeated deviation raises PRICE_ANOMALY for admin review. A single high quote never triggers automatic punishment.
4. Material is quoted as an estimate. If actual material exceeds the estimate by more than a configurable tolerance, the worker must submit an ADDITIONAL quote before billing it.
5. The customer sees the total and the split, with no hidden FIXO charges. A booking fee, if any, is shown separately and before submission.

Whether the customer must approve every quote in-app, or approval can be skipped for small amounts, is **OBD-4**. Recommendation: mandatory in V1.

## H. Job lifecycle and cancellation matrix

A job is created when a quote is accepted. It tracks physical execution only; money has its own status (section I).

| Job status | Meaning | Next states |
|---|---|---|
| CONFIRMED | Quote accepted; slot agreed | ON_THE_WAY, CANCELLED_* |
| ON_THE_WAY | Worker left for the address | ARRIVED, CANCELLED_*, NO_SHOW_WORKER |
| ARRIVED | Worker at the address (geofence check and/or customer start code) | IN_PROGRESS, CANCELLED_*, NO_SHOW_CUSTOMER |
| IN_PROGRESS | Work started | COMPLETION_SUBMITTED, CANCELLED_* |

| Job status | Meaning | Next states |
|---|---|---|
| COMPLETION_SUBMITTED | Worker entered final amount, summary, optional photo | COMPLETED, IN_PROGRESS (customer rejects completion) |
| COMPLETED | Customer confirmed, or auto-confirmed after the review window with no complaint | Terminal |
| CANCELLED_BY_CUSTOMER / _BY_WORKER / _BY_ADMIN | Cancelled with stage, reason and actor recorded | Terminal |
| NO_SHOW_WORKER / NO_SHOW_CUSTOMER | Missed appointment after grace period | Terminal |

**Rules:**

1. Every transition writes `job_status_history` with actor, timestamp, coordinates (where the app has them) and reason.
2. The final amount cannot exceed accepted quote plus accepted additional quotes without being flagged for admin review.
3. Auto-confirmation after the review window (default proposal: 24 hours) is configurable. A raised complaint pauses it.
4. A suspended worker's active jobs are not silently dropped: they go to an admin queue and are reassigned or completed under supervision (edge case 16).
5. Offline workers and closed apps do not break a job: the status is server-side, and the app resyncs on reconnect (edge cases 22–23).

**Cancellation matrix.** Each stage resolves a configurable rule (penalty points, fee, refund of lead access). Amounts are **OBD-8**; the structure below is fixed.

| Stage | Customer cancels | Worker cancels |
|---|---|---|
| Before any worker has access | Free. Leads withdrawn, no charges. | Not applicable (nothing accepted). A worker declining is just a declined lead. |
| Quote stage (access granted, no acceptance) | Free. Workers' lead fees follow the refund policy in OBD-2. | Withdraw quote. No penalty; frequent withdrawal feeds response-history ranking. |
| CONFIRMED, outside the free-cancel window | Free. | Penalty point; booking returns to MATCHING with priority; customer notified. |
| CONFIRMED, inside the window | Cancellation fee may apply (configurable, may be zero). | Larger penalty; lead fee credited back to the customer's next booking if configured. |
| ON_THE_WAY | Travel compensation to worker, configurable. | Strong penalty and admin flag; customer offered priority rebooking. |
| ARRIVED | Visit charge payable by customer. | Treated as a serious incident; admin review. |
| IN_PROGRESS | Pay for completed work; amount set by agreement or admin dispute decision. | Admin review; partial payment for work done if the customer agrees. |

Repeated cancellations by either side raise a flag (section T), never an automatic ban.

## I. Payment lifecycle

Payment status is a separate state machine from job status. A job can be COMPLETED while payment is still PENDING; neither field is derived from the other.

**Two records, not one:**

- `payment`: what is owed for a job or fee (amount due, currency, mode, status). One per payable event, for example the job total or a booking fee.

- `payment_transaction`: every attempt, gateway event, cash declaration, refund or adjustment. Append-only; never edited.

| Payment status | Meaning | Next states |
|---|---|---|
| NOT_DUE | Job not yet completed | PENDING |
| PENDING | Amount due; awaiting customer action | PROCESSING, PAID (cash), CANCELLED |
| PROCESSING | Gateway attempt in progress | PAID, FAILED |
| FAILED | Attempt failed; customer may retry | PROCESSING, CANCELLED |
| PARTIALLY_PAID | Part received (advance, or partial cash) | PAID, REFUND_PENDING |
| PAID | Amount due fully received | REFUND_PENDING, DISPUTED |
| REFUND_PENDING | Refund approved, not yet confirmed by gateway | PARTIALLY_REFUNDED, REFUNDED, FAILED |
| PARTIALLY_REFUNDED / REFUNDED | Money returned | Terminal |
| DISPUTED | Held for admin decision | PAID, REFUND_PENDING |
| CANCELLED | Cancelled before any money moved | Terminal |

**Online payment:**

1. Server creates the payment and a gateway order with the server-computed amount. The client cannot set the amount.
2. Customer pays through the gateway.
3. The gateway webhook is verified by signature, then processed once per gateway event ID. A repeated webhook finds the existing transaction and returns success without creating another.
4. Payment becomes PAID and the revenue lifecycle starts (section J).

5. A periodic reconciliation job compares the gateway's records with FIXO's, to catch missed webhooks.

**Cash payment:**

1. Worker marks "cash received" with the amount.
2. Customer confirms the amount in the app (or auto-confirms after a window with no objection).
3. A mismatch opens a dispute instead of settling.
4. FIXO's share of a cash job becomes a **commission due** entry on the worker's ledger account (see OBD-5 on how it is collected).

**Refunds** are admin or system initiated, always reference the original transaction, and are idempotent. Partial refunds are supported.

Whether payment is taken after the job (recommended for V1), partly in advance, or held until the customer confirms is **OBD-9**.

## J. Revenue lifecycle

Every financial fact is a row in an **append-only ledger**. Balances, earnings dashboards and reports are computed from the ledger. A mistake is fixed with a reversing entry plus a corrected entry, never by editing or deleting a row.

**Ledger structure:**

- `ledger_account` : one per worker (payable and commission-due), plus system accounts (gateway clearing, FIXO revenue, gateway fees, refunds, tax, customer credits).
- `ledger_entry` : transaction ID, account, debit or credit, amount in paise, currency, reference (booking, job, lead, payout, refund), the **rule version IDs** used (commission, lead fee, booking fee), actor, idempotency key, timestamp.
- Each transaction's debits equal its credits, enforced by a database constraint and a test.

**Lifecycle of money for one job:**

1. **Rules resolved** at booking creation and stored as a snapshot.
2. **Lead fee** (if the model uses one) is posted when contact access is granted.
3. **Payment PAID** triggers the settlement calculation using the snapshot, not today's rates.

4. **Earning accrued** to the worker as pending.
5. **Settlement hold** (configurable) lets refunds and disputes surface before money is released.
6. **Payout batch** moves eligible earnings to PAID_OUT through the payout provider; failures are retried and never double-pay (idempotent).
7. **Adjustments and refunds** post as new entries referencing the originals.

**Worked example from your brief** (illustrative; the 15% rate is only what ₹800 → ₹680 + ₹120 implies):

| Step | Debit | Credit | Amount (₹) |
|---|---|---|---|
| Customer pays online | Gateway clearing | Job collections | 800 |
| Worker share | Job collections | Worker payable | 680 |
| FIXO commission | Job collections | FIXO commission revenue | 120 |
| Gateway fee (if charged to FIXO) | Gateway fee expense | Gateway clearing | per gateway |
| Payout to worker | Worker payable | Payout clearing | 680 |

For a **cash job** there is no customer payment through FIXO. The ledger instead records a *commission due* entry against the worker, which is recovered according to **OBD-5**.

The worker dashboard shows exactly: **Gross → Fees → FIXO commission → Net**, each line traceable to ledger entries.

Who bears the gateway fee, whether GST applies to FIXO's fees and commission, and how invoices are issued are finance questions to confirm with an accountant (OBD-7).

## K. Database entities and relationships

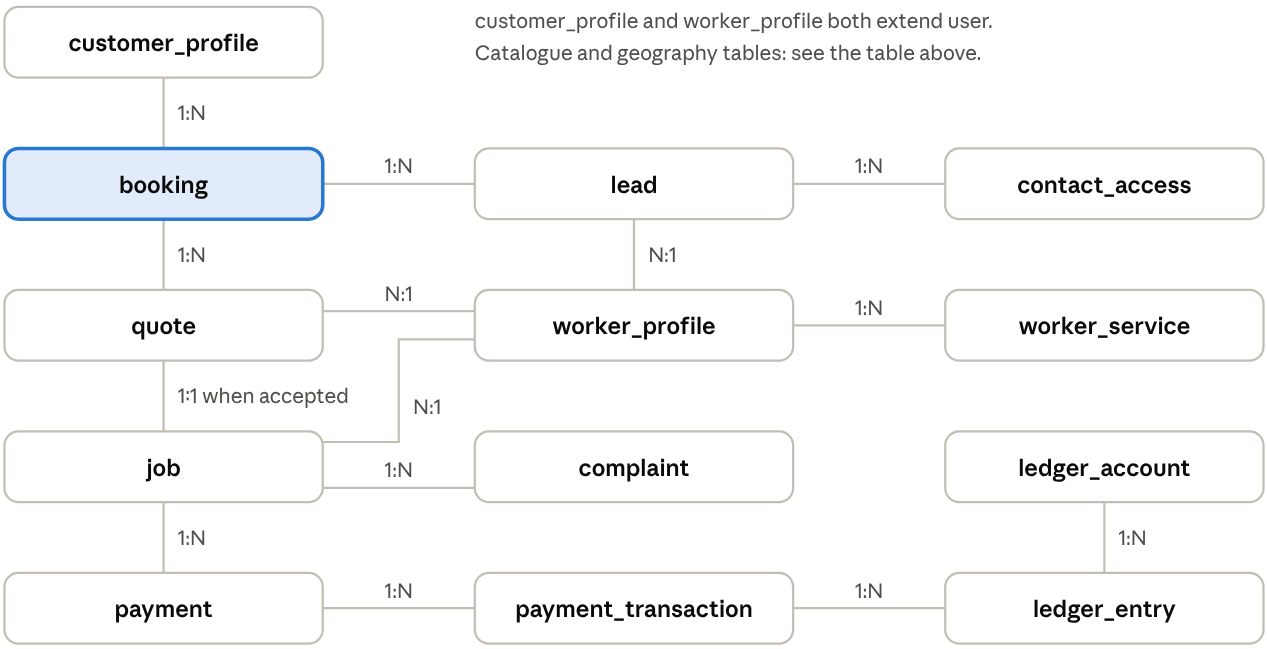
The model is normalised by domain. Conventions: UUID primary keys; money as integer paise; UTC timestamps; `city_id` on every operational table; status fields change only through services, each with a history table; no hard deletes for anything financial or audit-relevant.

| Domain | Entities | Key relationships and decisions |
|---|---|---|
| Identity | `user`, `customer_profile`, `worker_profile`, `admin_profile`, `admin_role`, `device`, `login_event` | One `user` per phone number. Profiles are 1:1 extensions of `user`. Admin permissions come from roles, not flags on the user. |
| Geography | `country`, `state`, `city`, `area`, `service_zone` | City → Area; a zone groups areas or holds a polygon. Admin-managed; workers and customers reference IDs, never free text. |
| Catalogue | `service_category`, `service_subcategory`, `service_problem`, `problem_requirement` | Category → Subcategory → Problem. Requirements link a problem to required skills/tools. Duration and emergency flags live on the problem. |
| Worker capability | `worker_service`, `worker_area`, `worker_availability`, `worker_state` | `worker_service` (worker × subcategory) carries its own skill-verified status. `worker_area` flags preferred areas and the travel limit. `worker_state` holds online flag and active-job count. |
| Verification | `kyc_verification`, `kyc_document`, `skill_verification` | Identity and skill are separate records with separate reviewers, expiry dates and decisions. Documents sit in private storage; only references are stored. |
| Booking | `booking`, `booking_problem`, `booking_media`, `booking_status_history`, `booking_rule_snapshot` | A booking has one or more problems. The snapshot stores the rule versions in force at creation. |

| Domain | Entities | Key relationships and decisions |
|---|---|---|
| Leads | `lead_pool`, `lead`, `lead_response`, `contact_access`, match_log | One pool per booking (cap, expiry, granted counter). One lead per booking–worker pair. Contact access is its own record. |
| Quotes | `quote`, `quote_line` | Quote revisions are immutable rows. Lines (visit, labour, material, discount) are separate rows; the total is computed server-side. |
| Jobs | `job`, `job_status_history`, `job_event` | One active job per booking. Events store timestamps and coordinates for arrival and completion. |
| Payments | `payment`, `payment_transaction` | Payment is separate from job. Transactions are append-only and keyed by gateway event ID. |
| Finance | `ledger_account`, `ledger_entry`, fee_calculation, `worker_payout`, `payout_item` | Earnings and commission are derived from ledger entries. See note below. |
| Configuration | `pricing_rule`, `revenue_rule`, `cancellation_rule`, `marketplace_setting`, `ranking_weight` | All versioned: scope (city, zone, category, plan), `effective_from`, `effective_to`, `version`, `created_by`. |
| Trust | `rating`, `complaint`, `dispute`, `dispute_evidence`, `penalty_event`, `fraud_flag` | Ratings are per party and dimension. Penalties and flags are events, which keeps decisions explainable. |
| Communication | `notification`, `notification_delivery`, `device_token` | One notification, many delivery attempts across channels. |

| Domain | Entities | Key relationships and decisions |
|---|---|---|
| Platform | `audit_log`, `idempotency_key`, `outbox_event` | The outbox pattern guarantees events are published exactly as committed. |

**Five records form the spine; workers, leads and money branch off**




core entity relationships · 13 entities, spine plus branches

The booking sits at the centre. Cardinalities read left to right as one-to-many (1:N), many-to-one (N:1) or one-to-one (1:1).

**Where this model deliberately differs from your entity list:**

- **Commission** is not a table. It is the combination of a versioned `revenue_rule`, the booking's rule snapshot and ledger entries, so history cannot be rewritten.
- **Review** is merged into `rating` (scores plus optional text) to avoid two tables for one event.
- **WorkerEarning** is not stored as a mutable balance. It is a view over ledger entries; a cached summary table may be added later for speed.
- **BookingProblem** allows multiple problems per booking, but V1 UI can start with one.
- **Notification** is split from delivery so retries and channel fallbacks are recorded cleanly.

# L. API architecture

REST over JSON, versioned under `/v1`, one route group per domain. Realtime events (section N) complement the API; they never replace it. A full OpenAPI specification will be generated in Phase 1 and kept as the contract.

| Group | Main endpoints | Who can call |
|---|---|---|
| `/auth` | `POST /otp/request`, `POST /otp/verify`, `POST /refresh`, `POST /logout` | Public (rate limited) |
| `/customers` | `GET/PATCH /me`, `GET/POST/PATCH/DELETE /me/addresses` | Customer |
| `/workers` | `GET/PATCH /me`, `PUT /me/areas`, `PUT /me/services`, `PUT /me/availability`, `POST /me/kyc/uploads`, `POST /me/submit`, `POST /me/online` | Worker |
| `/services` | `GET /categories`, `GET /categories/:id/problems`, `GET /search` | Public (cached) |
| `/areas` | `GET /` (active only), `POST /resolve` (coordinates → city, area, zone) | Public |
| `/bookings` | `POST /`, `GET /`, `GET /:id`, `POST /:id/cancel`, `POST /:id/rebook` | Customer (own) |
| `/leads` | `GET /`, `GET /:id`, `POST /:id/respond` | Worker (own) |
| `/quotes` | `POST /bookings/:id/quotes`, `POST /jobs/:id/quotes` (additional), `POST /:id/accept`, `/reject`, `/request-revision`, `/withdraw` | Worker submits; customer decides |
| `/jobs` | `GET /:id`, `POST /:id/transition`, `POST /:id/complete`, `POST /:id/confirm`, `POST /:id/cancel` | Worker and customer, by transition |

| Group | Main endpoints | Who can call |
|---|---|---|
| `/payments` | `POST /`, `POST /:id/cash-declare`, `POST /:id/cash-confirm`, `POST /webhooks/:provider` | Customer, worker; webhook is signature-verified |
| `/earnings` | `GET /summary`, `GET /ledger`, `GET /payouts` | Worker (own) |
| `/ratings` | `POST /jobs/:id/ratings`, `GET /workers/:id/ratings` | Parties to the job; public read of aggregates |
| `/complaints` | `POST /`, `GET /`, `POST /:id/evidence`, `POST /:id/messages` | Customer, worker (own) |
| `/notifications` | `GET /`, `POST /:id/read`, `PUT /devices/:token` | Authenticated |
| `/admin/*` | `workers`, `kyc`, `skills`, `catalogue`, `geo`, `rules`, `leads`, `jobs`, `payments`, `payouts`, `complaints`, `analytics`, `audit` | Admin roles only, by permission |

**Rules that apply to every endpoint:**

1. **Validation:** each request is validated against a schema; unknown fields are rejected. Prices, totals, commission, earnings, role, KYC status and payment status are never accepted from clients.
2. **Authentication:** short-lived access token plus rotating refresh token; every route declares the roles and permissions it needs.
3. **Authorization:** ownership is checked on every object (a customer can only read their own bookings).
4. **Idempotency:** create-booking, accept-quote, respond-to-lead, create-payment, refund and payout require an `Idempotency-Key` header. A repeat returns the stored result.
5. **Errors:** one shape, `{ error: { code, message, details, requestId } }`, with stable machine codes such as `LEAD_FILLED`, `QUOTE_EXPIRED`, `OUT_OF_SERVICE_AREA`, `PRICE_OUT_OF_RANGE`.
6. **Rate limits (starting values, tunable):** OTP request 3 per phone per 10 minutes plus per-IP and per-device caps; booking creation 5 per hour per customer; lead respond 30 per minute per worker; admin endpoints per user.
7. **Pagination:** cursor-based for all lists.

**Two worked contracts:**

- `POST /v1/bookings` takes `problem_ids`, `address_id` or `{lat, lng, house, landmark, pincode}`, `slot` (ASAP or date + window ID), `media_ids`, `notes`. It returns the booking ID, status REQUESTED, the resolved area and zone, the suggested price range and any booking fee. It fails with `OUT_OF_SERVICE_AREA`, `DUPLICATE_BOOKING_SUSPECTED` (with the existing booking ID) or `SLOT_UNAVAILABLE`.
- `POST /v1/leads/:id/respond` takes `{ action: "interested" | "ignore" | "unavailable" }`. For `interested` it returns the lead status and the contact channel, or fails with `LEAD_FILLED`, `LEAD_EXPIRED`, `INSUFFICIENT_CREDIT` or `WORKER_NOT_ELIGIBLE`.

## M. Authentication and authorization

### Authentication

- **Customers and workers:** mobile number + OTP. Google login can be added for customers later without changing the model, because identity is a `user` row with linked auth methods.
- **Admins:** email + password with mandatory multi-factor authentication, separate login surface, shorter sessions.
- **Tokens:** access token (about 15 minutes) and rotating refresh token stored hashed server-side, bound to a device record. Reuse of an old refresh token revokes the whole session family.
- **OTP abuse prevention:** per-phone, per-IP and per-device limits; cooldowns that grow with repeated requests; short expiry; attempt limits; no differing responses that reveal whether a number exists; a provider-level fallback channel.

### Roles and who can be what

One `user` can hold more than one role (a worker can also book a service as a customer). The app context decides which role a request acts as, and every token carries the active role.

| Role | Gate for normal actions |
| --- | --- |
| Customer | Verified phone |

| Role | Gate for normal actions |
|---|---|
| Worker | Verified phone **and** worker status VERIFIED to receive leads; REGISTERED to SUSPENDED states restrict features progressively |
| Admin | Active admin profile + MFA + permission set |

**Authorization has three layers**

1. **RBAC:** each route declares the permission it needs. Admin roles (super admin, operations, verification, finance, support) hold different permission sets, for example only finance can approve payouts and only super admin can change revenue rules.
2. **Ownership checks:** a customer reads only their bookings; a worker reads only leads offered to them and jobs they hold; every query is scoped by the owner.
3. **State guards:** actions are valid only in the right state (a lead can accept access only while OFFERED/VIEWED, a worker must be VERIFIED and not suspended, a job transitions only from its allowed predecessors).

**Other controls**

- Customer details shown to workers are limited by `contact_access` status, which the server checks per request.
- KYC and job media are served only through short-lived signed URLs after an authorization check.
- Payment webhooks authenticate by provider signature, not by user token.
- Every admin action that changes money, rules, KYC or account status writes an audit row (section T).
- Suspended or blocked users have sessions revoked and sockets disconnected.

## N. Realtime architecture

Realtime is used only for events where seconds matter. Everything else is fetched normally. The socket is a delivery hint; the database and REST API remain the source of truth, so a missed event never corrupts state.

| Event | Recipient | Why realtime | If the app is closed |
|---|---|---|---|
| `lead.offered` | Worker | Leads are time-sensitive | Push notification |
| `lead.filled` / `lead.expired` | Worker | Remove stale leads from the list | Next sync |
| `booking.worker_interested`, `quote.received` | Customer | Live waiting screen | Push |
| `quote.decided` | Worker | Immediate confirmation | Push |
| `job.status_changed` | Customer, worker | Tracking | Push for key steps |
| `payment.updated` | Customer, worker | Instant confirmation | Push |
| `account.status_changed` | Worker, admin | Suspension or approval takes effect at once | Next request returns the error |
| `admin.exception_raised` | Admin | Live queue counts | Dashboard refresh |

**How duplicates and loops are prevented:**

1. **One socket per app instance.** The connection is opened once at app level after login, never per screen. Screens read from a shared store, so navigating cannot create duplicate subscriptions.
2. **Server-assigned rooms.** On connect, the server joins the user to `user:{id}` (and booking rooms the user is a party to). Clients do not choose rooms.
3. **Event identity.** Every event has a unique ID and a per-room sequence number, produced from the transactional outbox. Clients ignore IDs already seen.
4. **Gap recovery.** On reconnect the client sends its last sequence and receives missed events from a REST catch-up endpoint, or refetches the affected object.
5. **Reconnect control.** Exponential backoff with jitter, a capped retry count, and a resync on foreground. A token refresh happens before reconnect, so an expired token does not cause a loop.

6. **Notification de-duplication.** A socket event and its push share a `dedupe_key`; the client shows one.
7. **Horizontal scale.** Redis pub/sub adapter shares rooms across API instances.
8. **Security.** Sockets authenticate with the same access token; suspended users are disconnected by an event; payloads carry only what the recipient is allowed to see.

## O. Notification architecture

A notification is created once from a domain event, then delivered through the cheapest channel that works. Delivery attempts are recorded separately so failures can be retried and reported.

**Pipeline:** domain event (from the outbox) → notification rules (who, which template, which channels, priority) → preferences and quiet-hours check → de-duplication and throttling → delivery job per channel → delivery result stored → fallback if needed.

**Channel policy:**

| Priority | Examples | Channels, in order |
|---|---|---|
| Critical, time-bound | New lead for worker, worker on the way, OTP | Push → (if undelivered after a short wait) SMS |
| Important | Quote received, job confirmed, payment received, KYC decision | Push → WhatsApp or SMS where the user opted in |
| Informational | Rating reminder, weekly earnings summary | Push or email, batched |
| Marketing | Offers | Opt-in only; never mixed with transactional channels |

**Rules to avoid spam:**

1. Each template has a throttle (for example, at most one "lead expiring" reminder per lead).
2. Workers can mute categories and set quiet hours, except critical items that carry a time-limited action.
3. Related events are batched into one message (three quotes in one minute become one "3 new quotes").
4. Notifications for a lead disappear when the lead is filled or expired.

5. A `dedupe_key` prevents the same event notifying twice, even after a retry.
6. WhatsApp and SMS templates are pre-approved with the provider, stored with version numbers, and sent in the user's chosen language.

**Failure handling:** a failed push triggers the fallback channel; repeated failures mark a device token invalid; undeliverable critical notifications raise an admin exception (edge case 24). Delivery status is visible in the admin user view for support.

## P. Pricing engine

The pricing engine guides and checks prices; it does not force a single fixed price, because real jobs vary. It exists to solve overcharging, inflated visiting charges and customer price uncertainty.

**Pricing guideline (one per problem and scope, admin-managed, versioned):**

| Component | Meaning | Used for |
|---|---|---|
| Visit fee: cap and default | Maximum a worker may charge to attend | Stops inflated visiting charges |
| Labour: minimum, suggested, maximum | Expected range for the work | Quote validation and customer expectations |
| Material policy | Actual cost, approved range, or fixed list | Prevents surprise material bills |
| Emergency fee | Added for emergency bookings | Transparent urgent pricing |
| Distance fee | Per-km fee beyond the normal radius, set by FIXO rule | Replaces worker-invented travel charges |
| Customer booking fee | Optional FIXO fee, shown to the customer before submit | Revenue (if chosen) |

**Scope resolution:** the most specific active rule wins: problem + zone → problem + city → subcategory + city → category + city → global default. Each rule has `effective_from` and `effective_to`; changes create a new version.

**Snapshot:** on booking creation the resolved guideline version is stored. Old bookings are always evaluated against the rules that applied then.

**Customer-facing price:** before booking, the customer sees the suggested range, the visit fee cap, whether material is extra and any emergency or distance fee. After booking, quotes appear against that range.

**Quote validation:** the server classifies each quote line.

| Result | Condition | System response |
|---|---|---|
| IN_RANGE | Within guideline | Accepted normally |
| BELOW_MIN | Under the minimum | Allowed with a warning (protects quality) |
| ABOVE_SUGGESTED | Over suggested but within the maximum | Allowed; shown to the customer as above typical |
| ABOVE_MAX | Over the maximum | Worker must add a justification; flagged; admin-configurable to block outright |
| VISIT_OVER_CAP | Visit fee above the cap | Rejected by the server |

**PRICE_ANOMALY detection:** a scheduled job scores each worker on the share of recent quotes classified ABOVE_MAX or VISIT_OVER_CAP, relative to peers in the same category and zone. When the score exceeds a configurable threshold over a configurable window, a PRICE_ANOMALY flag is raised for admin review. Admin options: warning, ask for justification, reduce lead priority, temporary lead limit, suspension, removal. **A single quote never triggers automatic punishment.**

The distance fee, visit cap and matching radius work together: nearby workers are preferred (section Q), the radius is configured per zone, and the worker sees the customer's area before accepting.

## Q. Worker matching algorithm

Matching runs in two stages: **hard filters** decide who is eligible, then a **weighted score** ranks the eligible workers. Your priorities 1–4 are filters (a worker either qualifies or does not); priorities 5–9 are score factors.

**Stage 1: hard filters (all must pass)**

1. Account VERIFIED and not suspended or blocked; KYC not expired.

2. Skill verified for the requested subcategory (see OBD-12 for a probation exception).
3. Covers the booking's area, or lies within the worker's own maximum travel distance and the zone's lead radius.
4. Available at the requested time: for ASAP, online and below `max_active_jobs`; for scheduled, inside working hours with no conflicting job.
5. Not blocked by this customer, and not the customer.
6. Not under an active lead restriction (for example from a PRICE_ANOMALY decision).

**Stage 2: score (each factor normalised to 0–1)**

```
score = \sum_i w_i \cdot f_i, \quad \sum_i w_i = 1
```

| Factor | How it is measured | Note |
|---|---|---|
| Distance | Road distance or travel time to the address, falling back to straight-line distance | Closer scores higher; prefers nearby workers to reduce visit charges |
| Rating | Bayesian-smoothed average | Avoids a new worker with one 5-star rating outranking a proven one |
| Completed jobs | Log-scaled count in this category | Diminishing returns |
| Response history | Response rate and median time on recent leads | Rewards workers who answer |
| Workload | Active jobs against maximum | Spreads work, avoids overload |
| Cancellation record | Recent cancellations and no-shows | Negative factor |
| Price fairness | Share of recent quotes in range | Negative if flagged |
| Fairness boost | Time since last offered lead; new-worker boost | Prevents a few workers taking everything |

Weights live in `ranking_weight`, scoped by city, zone and category, and are versioned. The weights of the plan tiers (Basic, Pro, Premium) can later be added as a factor without redesigning matching.

**Process for each booking**

1. Resolve zone and rules; store the snapshot.
2. Query candidates with PostGIS using the area and radius.
3. Apply hard filters, then score and sort.
4. Write the ranked list with each factor's value to `match_log`, so admins can explain any decision.
5. Release leads in waves (section F); reserve at least one slot per wave for exploration (a newer or less-offered qualified worker) if admin enables it.
6. If nobody qualifies, widen the radius in configured steps, then move to NO_WORKER_AVAILABLE and raise an admin exception.

**Emergency bookings** use a larger radius, skip wave delays and raise the weight on proximity and response time.

**Privacy:** workers' live location is used only while they are online and have consented; otherwise matching uses their registered base location.

## R. Revenue engine

The revenue engine turns a business event into fee records using versioned rules. No rate, fee or model is written in code. Models A–E from your brief are all expressed as configuration of the same engine.

**A revenue rule defines:** scope (global, city, zone, category, worker plan), fee type, trigger, amount, limits, effective dates and version.

| Fee type | Calculation | Trigger | Paid by |
|---|---|---|---|
| Lead fee | Fixed amount, optionally per category or zone | Contact access granted | Worker (cash or prepaid credit) |
| Commission | Percentage, fixed or tiered, with optional minimum and maximum | Payment PAID, or job COMPLETED for cash | Worker (deducted from earning) |
| Customer booking fee | Fixed or percentage | Booking confirmed, or created | Customer, shown before submit |
| Subscription | Plan price and included lead allowance, with overage fee | Billing cycle | Worker (later phase) |

| Fee type | Calculation | Trigger | Paid by |
|---|---|---|---|
| Surcharge share | Share of emergency or distance fee | Job completed | Split between worker and FIXO |

**Evaluation:** the engine receives (event, booking snapshot, worker plan), picks the most specific active rule, calculates, and writes a `fee_calculation` record with inputs, rule version and output. The ledger entries reference that record. The same input always gives the same result, and re-running an old booking uses its snapshot rather than today's rule.

**Mapping of your five models**

| Model | Configuration | Main strength | Main risk |
|---|---|---|---|
| A. Pay per lead | Lead fee on access, commission 0 | Revenue before the job; independent of leakage | Workers pay without a guaranteed job |
| B. Prepaid credits | Lead fee deducted from a credit balance | Upfront cash; fast response; no per-lead payment friction | Needs wallet top-up flow; refund rules |
| C. Subscription | Plan includes lead allowance | Predictable revenue | Premature before there is steady lead volume |
| D. Commission | Percentage of completed job value | Aligned with success | Hard to collect on cash jobs; off-platform leakage |
| E. Hybrid | Any mix, for example small lead fee + commission on online jobs | Flexible | More complex to explain to workers |

**Recommendation for launch (OBD-1):** start with B, prepaid lead credits with a small fee, plus the lead cap and a refund if the customer is unreachable. It brings revenue without needing every payment to go through FIXO, which is realistic in a cash-heavy local market. Collect data on lead-to-job conversion, then add commission on online-paid jobs as in-app payment adoption grows. Because both are rules in the same engine, this is a configuration change, not a rebuild.

**Guardrails:** every rule change needs a reason, is versioned with an effective date, appears in the audit log, and can be previewed against the last 30 days of bookings to show the revenue impact before it goes live.

## S. Complaint and dispute system

A **complaint** is a report from either party. A **dispute** is a complaint that needs an admin decision about money or service. Many complaints are resolved by support without becoming disputes.

**Complaint types**

- Customer: worker did not arrive, overcharged, poor work, behaviour, work incomplete, payment issue, other.
- Worker: customer unavailable, wrong information, unsafe location, payment issue, customer abuse, other.

**Status flow:** OPEN → UNDER_REVIEW → (AWAITING_INFO ↔ UNDER_REVIEW) → RESOLVED → CLOSED. A complaint can be ESCALATED to a dispute, and a dispute can be REOPENED once within a configurable window.

**What the system does automatically when a dispute opens:**

1. Sets payment to DISPUTED and holds the related earning and payout.
2. Pauses job auto-confirmation.
3. Freezes the evidence bundle: quotes and revisions, job status history with timestamps and coordinates, media, notification log, masked-call metadata and chat (not call audio), rule snapshot and ratings.
4. Starts the response timer for the other party.
5. Notifies both parties.

**Admin resolution options (each requires a reason and is audited):**

| Outcome | Effects |
|---|---|
| Dismiss | No action; hold released |
| Warn | Warning recorded against the party |
| Redo work | Job reopened for rework, optionally with the same worker |

| Outcome | Effects |
|---|---|
| Partial or full refund | Refund transaction and ledger adjustment; worker earning reduced accordingly |
| Worker penalty | Penalty points, lead limits, suspension, or removal |
| Customer restriction | Booking limits or block |
| Goodwill credit | Credit to customer from FIXO revenue |

**Targets and tracking:** first response time and resolution time targets are configurable; overdue items appear in the admin exception queue. Complaint rate by worker, category and area feeds the quality score and the fraud signals (section T).

**Protecting both sides:** workers can contest a complaint with evidence; a worker rating given by a customer is never deleted because of a dispute, but an admin can mark it as disputed or remove it for proven abuse (with an audit reason). A worker's rating of a customer is private to FIXO and not shown publicly.

## T. Security architecture and anti-leakage

### Security controls

| Area | Control |
|---|---|
| Transport and storage | TLS everywhere; encryption at rest for database, backups and buckets; field-level encryption for KYC identifiers and bank details |
| KYC and media | Private buckets with random object keys; signed, short-lived URLs issued only after an authorization check; every KYC document view is logged with who and why; upload limits on type and size, content sniffing and malware scan |
| Identity numbers | Prefer verification through a licensed KYC provider and store the result and a masked reference rather than the full number (confirm legal requirements, OBD-13) |

| Area | Control |
|---|---|
| Secrets and access | Secrets in a managed vault; least-privilege database roles; separate production and staging; no personal data in logs |
| Input and API | Schema validation, rejection of unknown fields, parameterised queries, output encoding in admin UI, CSRF protection where cookies are used, strict CORS |
| Money | Server-only calculation; ledger append-only with balanced entries; idempotency keys; webhook signature checks; daily reconciliation against the gateway |
| Admin | MFA, role permissions, session limits, IP allow-listing as an option, an audit log that cannot be edited through the application |
| Operations | Automated backups with restore drills, dependency scanning, monitoring and alerting on error rate, queue depth, failed payments and OTP spikes |
| Privacy | Collect only what a feature needs; defined retention periods; consent records; worker and customer data export and deletion paths. Confirm Indian data-protection obligations with counsel before launch |

### Fraud and abuse prevention

| Threat | Signals tracked | Response |
|---|---|---|
| Fake or duplicate worker accounts | Same device, bank account, document hash, face match, phone patterns | Block at KYC; flag for admin |

| Threat | Signals tracked | Response |
|---|---|---|
| Fake bookings and lead farming | Many bookings without completion, pattern of cancel after contact, out-of-area addresses, same device across customers | Rate limits, booking restrictions, manual review |
| Repeated cancellations | Cancellation rate by party | Warnings, lead limits, penalties |
| Fake reviews | Review only after a paid or confirmed job; unusual rating clusters | Hold review; admin review |
| Payment fraud | Chargebacks, repeated failed attempts, mismatched cash declarations | Hold payout; restrict payment methods |
| Referral abuse | Same device or payout account across referrals | Withhold reward |

Device and IP data is kept to what fraud detection needs, with a stated retention period.

**Customer–worker off-platform leakage**

Hiding phone numbers helps only briefly. The design goal is that staying on FIXO is better for both sides.

- **Gated, masked contact:** workers never see the customer number by default; a masked call or chat is granted through `contact_access` and expires.
- **What the customer gets by staying:** price transparency, a written quote, verified identity, payment protection and dispute help, warranty or service-history records, receipts, and **Book again** in one tap. These are lost off-platform.
- **What the worker gets by staying:** a steady stream of leads, a reputation that carries to the next customer, in-app payment with a clear earnings view, and, in the future, lower fees for repeat on-platform customers.
- **Chat guardrails:** in-app chat can detect phone numbers and obvious attempts to move off-platform, warning first and flagging repeats.
- **Detection signals:** a customer–worker pair with an accepted quote, no completed job and then silence; workers with an unusual share of cancelled-after-contact jobs; customers whose bookings drop after specific workers.

- **Response is incentives first, penalties second:** repeat-booking rewards, priority for workers with high on-platform completion, and warnings or lead limits only for clear patterns.

What happens after a first connection is the central business risk (your section 43), and the metrics for it appear in section 57 analytics and in the admin dashboard.

## U. MVP, V2 and V3 separation

The MVP builds the whole marketplace loop once, correctly, with the foundations (configuration, ledger, status history, audit) that are expensive to retrofit. Features that are easy to add on top are deferred.

| Area | MVP | V2 | V3 |
|---|---|---|---|
| Accounts | Customer and worker OTP login; admin with MFA | Google login; multi-role switcher polish | Organisations or teams of workers |
| Catalogue and geography | Admin-managed categories, problems, zones; Lucknow active | Seasonal and bundled services | Multi-city, multi-state rollout tools |
| Worker onboarding | Profile, areas, services, KYC upload, manual KYC and skill review | Automated KYC checks, KYC expiry reminders | Training and certification programs |
| Matching | Hard filters plus weighted score; waves; cap | Advanced ranking, predictive matching | Marketplace optimisation, dynamic radius |
| Leads and contact | Lead offers, cap, masked call | In-app chat and richer contact options | Smart lead allocation |
| Quotes and pricing | Structured quotes, guideline ranges, anomaly flags | Smart pricing suggestions | Dynamic pricing |

| Area | MVP | V2 | V3 |
|---|---|---|---|
| Jobs | Full lifecycle with history, start code, geofence check where available | Reschedule flows, multi-visit jobs | Automated scheduling |
| Payments | Online payment, cash recording, refunds by admin | Wallet, partial payments and advances | Escrow-style holds if needed |
| Revenue | Configurable rules, ledger, lead credits, basic revenue reports | Subscriptions, plan tiers | Automated plan optimisation |
| Payouts | Admin-approved payout batches | Automated payouts | Instant payouts |
| Trust | Ratings, complaints, disputes, manual admin resolution | Advanced fraud detection, AI-assisted support | Automated dispute resolution, worker performance scoring |
| Notifications | Push, SMS, WhatsApp for key events | Preference centre, richer templates | Smart timing |
| Admin and analytics | Core management, funnel, revenue, exceptions queue | Cohorts, advanced analytics, CAC dashboards | Forecasting |
| AI | None in the critical path | AI support assistant | AI service diagnosis from photos |

**Deliberately not in the MVP:** subscriptions, wallet top-ups beyond lead credits, automated payouts, in-app chat, dynamic pricing, multi-city, AI features, referral programs. The architecture leaves a clear place for each, so adding one later does not change the core model.

## V. Edge cases and their state transitions

Each case below has a defined detection, handling and resulting state. None of them relies on someone remembering to fix it by hand; anything unresolved lands in the admin exceptions queue.

| # | Case | Handling | Resulting states |
|---|---|---|---|
| 1 | No worker available | Radius widens in configured steps, then retry window; customer offered scheduled slot or cancel; admin alerted | Booking MATCHING → NO_WORKER_AVAILABLE → MATCHING / CANCELLED / EXPIRED |
| 2 | Only one worker available | Lead proceeds; customer told only one option exists, can wait for more or accept | Booking INTERESTED → QUOTED as normal |
| 3 | Worker accepts then cancels | Before quote acceptance: slot freed and next wave released; after: job cancelled with penalty per matrix and rematch with priority | Lead WITHDRAWN; or Job CANCELLED_BY_WORKER, Booking CONFIRMED → MATCHING |
| 4 | Customer cancels | Matrix by stage; leads withdrawn; contact access revoked; lead-fee refunds per OBD-2 | Booking CANCELLED; leads WITHDRAWN; Job CANCELLED_BY_CUSTOMER if one exists |
| 5 | Customer does not answer | Worker logs unreachable attempts (call metadata recorded); after configured attempts the lead fee is refunded and the booking flagged | Lead stays ACCESS_GRANTED with `unreachable` flag; customer flag count rises |
| 6 | Worker does not answer | Customer taps "worker not responding"; after timeout the slot is freed and the next wave released | Lead WITHDRAWN (system); response history updated |
| 7 | Worker arrives late | Late beyond grace allows free customer cancellation and records lateness | Job stays or CANCELLED_BY_CUSTOMER with no fee; lateness feeds ranking |

| # | Case | Handling | Resulting states |
|---|---|---|---|
| 8 | Wrong customer location | Pin and address compared at booking; at the door the worker reports wrong location, job pauses, customer corrects within a window; extra distance becomes an additional quote | Job ARRIVED (paused) → continues, or NO_SHOW_CUSTOMER |
| 9 | Worker too far | Excluded by radius; admin override applies the distance fee rule and shows distance to the worker beforehand | Lead not created, or created with distance fee quote |
| 10 | Price higher than expected | Flag and justification; customer sees above-typical label and can reject or ask for revision | Quote SUBMITTED / REVISION_REQUESTED / REJECTED |
| 11 | Additional work required | Worker submits ADDITIONAL quote; extra scope waits for acceptance | Job IN_PROGRESS with pending additional quote → accepted or original scope only |
| 12 | Material required | Material estimate in quote; overage beyond tolerance needs an additional quote | As above |
| 13 | Payment fails | Retry, switch method, or cash; reminders; admin queue after configured days | Payment PROCESSING → FAILED → PROCESSING / PAID (cash) |
| 14 | Payment succeeds, webhook repeats | Webhook deduplicated by gateway event ID; unique constraint; no new ledger entries | Payment stays PAID; duplicate event logged |
| 15 | Worker KYC expires | Warnings before expiry; on expiry the worker stops receiving new leads and enters re-review; active jobs continue | Worker VERIFIED → UNDER_REVIEW |

| # | Case | Handling | Resulting states |
|---|---|---|---|
| 16 | Worker suspended during active booking | Sessions revoked; open leads withdrawn; contact access revoked; active jobs go to an admin queue where admin chooses supervised completion or reassignment; payout held; customer told neutrally without the reason | Worker SUSPENDED; Jobs held → CANCELLED_BY_ADMIN with Booking → MATCHING (priority), or continue |
| 17 | Customer reports fraud | Complaint of type fraud escalates to a dispute; payment held; evidence frozen | Complaint → Dispute; Payment DISPUTED |
| 18 | Worker reports customer abuse | Safety flag; worker may leave without penalty; customer restricted pending review | Job CANCELLED_BY_WORKER (no penalty); customer restricted |
| 19 | Multiple workers accept at once | Pool row lock; the last slot goes to the first transaction; others see filled with no charge | Lead ACCESS_GRANTED vs CLOSED_FILLED |
| 20 | Lead expires | Timer sets EXPIRED; late taps rejected; next wave or NO_WORKER_AVAILABLE | Lead EXPIRED; Booking may move to NO_WORKER_AVAILABLE |
| 21 | Accidental duplicate booking | Same customer, problem, address in a short window prompts "open existing booking?"; retries are caught by idempotency; real duplicates are allowed | Existing booking returned, or a second booking created |
| 22 | Worker loses internet | Status taps queue on the device with timestamps and sync on reconnect; server accepts idempotently and flags late data; customer sees "worker offline" after a heartbeat gap | Job state changes on sync only |

| # | Case | Handling | Resulting states |
|---|---|---|---|
| 23 | App closes during active job | State lives on the server; on reopen the app resyncs; push reminders continue | No state change |
| 24 | Notification fails | Retry, then fallback channel; invalid tokens retired; critical failures raise an admin exception; in-app inbox always holds the item | Delivery FAILED → fallback delivery |
| 25 | Socket disconnects | Backoff reconnect; catch-up from the last sequence on reconnect or foreground; no continuous polling | No state change |

## W. Frontend screens

Customer screens are few and simple. Worker screens are built for speed. Admin screens are listed in section Y as modules.

**Customer app (Hindi and English)**

| # | Screen | Key content |
|---|---|---|
| C1 | Welcome and language | Language choice; no login wall |
| C2 | Home | Category grid, search, recent services, active bookings, help, emergency entry |
| C3 | Problem selection | Subcategories and problems for the chosen category |
| C4 | Problem detail | Suggested price range, visit cap, duration, material note |
| C5 | Photo or video | Optional upload with progress and retry |
| C6 | Location | Current location, saved address or map pin; service-area check |

| # | Screen | Key content |
|---|---|---|
| C7 | Time | ASAP, today, tomorrow, date; admin-defined windows |
| C8 | Review and submit | Summary, price guidance, booking fee, duplicate-booking prompt |
| C9 | OTP login | Phone, OTP, name only |
| C10 | Finding workers | Live status, wave progress, option to widen search or schedule |
| C11 | Worker options | Cards with rating, jobs, verification badges, distance, quote |
| C12 | Quote detail | Itemised quote, compare to range, accept, reject, ask for revision |
| C13 | Job tracking | Status steps, worker details, masked call, start code |
| C14 | Completion | Final amount, work summary, confirm or report issue |
| C15 | Payment | Online (UPI or card) or cash, with receipt |
| C16 | Rating | Five dimensions plus comment |
| C17 | My bookings | List and detail with full timeline and receipt |
| C18 | Book again | Prefilled booking with same worker or any worker |
| C19 | Addresses | Manage saved addresses |
| C20 | Profile and settings | Language, notifications |
| C21 | Help and complaints | Raise and follow a complaint |
| C22 | Notifications | Inbox |

### Worker app

| # | Screen | Key content |
|---|---|---|
| W1 | Login | Phone and OTP |

| # | Screen | Key content |
|---|---|---|
| W2–W9 | Onboarding | Profile; city and areas; services and skills; experience; KYC upload; bank or UPI; availability and pricing preferences; agreement |
| W10 | Verification status | Pending, rejected with reason and resubmit, verified |
| W11 | Home | Online toggle, lead inbox, today's jobs, earnings summary, credit balance |
| W12 | Lead detail | Problem, media, area, distance, time, suggested price, fee; Interested, Ignore, Not available |
| W13 | Lead access | Fee or credit deduction confirmation; result (granted or filled) |
| W14 | Contact | Masked call or chat; call-result logging (reached, unreachable) |
| W15 | Quote builder | Visit, labour, material, discount; live guideline check; justification field |
| W16 | Job detail | Address, customer, agreed price; status buttons |
| W17 | Completion | Final amount, summary, photo; additional quote |
| W18 | Payment | Cash declared or online status |
| W19 | Earnings | Day, week, month; gross → fees → commission → net; pending settlement |
| W20 | Credits | Balance, top-up, transaction history |
| W21 | Payouts | History and status |
| W22 | Ratings and profile | Own rating, reviews, profile preview |
| W23 | Availability and areas | Working hours, days, max jobs, areas, emergency toggle |
| W24 | Complaints | Raise and follow |
| W25 | Notifications and settings | Inbox, mute categories, quiet hours |

**Admin screens** correspond to the modules in section Y.

# X. Backend modules

Each module owns its tables and exposes a service interface. Other modules call that interface or react to its events; they never write another module's tables. This is what allows a module to become its own service later.

| Module | Owns | Key events published | Depends on |
|---|---|---|---|
| `auth` | Sessions, OTP, devices, tokens | `user.logged_in`, `session.revoked` | `users`, `notifications` |
| `users` | User, profiles, roles | `user.created`, `user.status_changed` | none |
| `geo` | Country to zone, polygons, resolution | `area.activated` | none |
| `catalogue` | Category, subcategory, problem, requirements | `catalogue.changed` | `config` |
| `workers` | Worker services, areas, availability, state | `worker.online_changed`, `worker.status_changed` | `geo`, `catalogue`, `verification` |
| `verification` | KYC, skill verification, expiry | `kyc.decided`, `skill.decided`, `kyc.expiring` | `media`, `workers` |
| `bookings` | Booking, history, snapshot | `booking.requested`, `booking.status_changed` | `catalogue`, `geo`, `pricing`, `config` |
| `matching` | Ranking, `match_log`, wave scheduler | `matching.completed`, `matching.exhausted` | `workers`, `geo`, `config` |
| `leads` | Lead pool, leads, responses, contact access | `lead.offered`, `lead.access_granted`, `lead.filled` | `matching`, `revenue`, `ledger`, `contact` |
| `contact` | Masked calls and chat metadata | `call.completed` | `leads`, telephony provider |

| Module | Owns | Key events published | Depends on |
|---|---|---|---|
| `quotes` | Quotes and lines | `quote.submitted`, `quote.accepted` | `pricing`, `bookings` |
| `jobs` | Job, history, events | `job.status_changed`, `job.completed` | `quotes`, `bookings` |
| `pricing` | Guidelines, quote validation, anomaly scoring | `price.anomaly_flagged` | `config` |
| `revenue` | Revenue rules, fee calculations | `fee.calculated` | `config`, `ledger` |
| `payments` | Payments, transactions, webhooks, refunds | `payment.paid`, `payment.failed`, `payment.refunded` | `jobs`, gateway |
| `ledger` | Accounts and entries; balance queries | `ledger.posted` | none (called by others) |
| `payouts` | Payout batches and items | `payout.completed`, `payout.failed` | `ledger`, payout provider |
| `ratings` | Ratings and aggregates | `rating.created` | `jobs` |
| `trust` | Complaints, disputes, penalties, fraud flags | `dispute.opened`, `dispute.resolved`, `penalty.applied` | `jobs`, `payments`, `ledger` |
| `notifications` | Templates, rules, deliveries, tokens | `notification.failed` | channel providers |
| `realtime` | Socket gateway and room logic | none | `auth`, outbox |
| `media` | Upload requests, signed URLs, scanning | `media.ready` | storage |

| Module | Owns | Key events published | Depends on |
|---|---|---|---|
| `config` | Marketplace settings, versioned rules registry | `config.changed` | none |
| `analytics` | Funnel events, aggregates | none | outbox (read-only consumer) |
| `audit` | Audit log writer | none | called by admin and finance actions |
| `platform` | Idempotency, outbox, job queue, scheduler, rate limiter | none | none |

**Rules for the codebase**

1. A module may import another module's public interface only; this is enforced by lint rules and tests.
2. State changes go through the owning service, never directly through the database.
3. Events are written to the outbox in the same transaction as the state change, then published by a worker process.
4. Scheduled work (lead expiry, wave release, auto-confirmation, KYC expiry, reconciliation, anomaly scoring) runs in the queue, with locks so a job runs once even with several instances.

# Y. Admin panel modules

Roles: **SA** super admin, **OPS** operations, **VER** verification, **FIN** finance, **SUP** support. Every change to rules, money, KYC or account status requires a reason and writes an audit row.

| # | Module | What admins do | Roles |
|---|---|---|---|
| 1 | Overview | Totals (customers, verified and active workers, bookings, completed and cancelled jobs, revenue, payouts, complaints), funnel conversion, today's exceptions | All (scoped) |
| 2 | Workers | Search and filter; profile, areas, services, jobs, earnings, complaints, ratings; verify, reject, suspend, block, restore | OPS, VER, SA |
| 3 | Verification queue | Review KYC documents; approve or reject with reason; separately mark skills per category; trial-job notes; expiry dates | VER, SA |
| 4 | Customers | Search; profile, bookings, complaints, payment history; restrict or block | SUP, OPS, SA |
| 5 | Bookings and leads | Booking timeline; lead pool state; reopen, withdraw, reassign, extend radius; view match log with factor scores | OPS, SUP |
| 6 | Jobs | Job timeline with coordinates; admin cancel or reassign; override stuck states | OPS |
| 7 | Catalogue | Create and edit category, subcategory, problem, requirements, duration, emergency flag; activate or retire | OPS, SA |
| 8 | Geography | Cities, areas, zones with map polygons; activate or deactivate; merge duplicate areas | OPS, SA |
| 9 | Pricing rules | Edit guidelines per problem and scope; versioned with effective dates; impact preview | OPS, SA |
| 10 | Revenue rules | Commission, lead fee, booking fee, plans; versioned; preview against recent bookings | SA (FIN to approve) |

| # | Module | What admins do | Roles |
|---|---|---|---|
| 11 | Marketplace settings | Lead cap, waves, expiry, radius, ranking weights, cancellation rules, slot windows, auto-confirm window | OPS, SA |
| 12 | Payments and refunds | Payment search; issue full or partial refund; handle failed payments | FIN |
| 13 | Ledger | Browse entries; reconcile with gateway; post adjustments with reversal rules | FIN |
| 14 | Payouts | Review payout batch; approve; retry failed; export | FIN |
| 15 | Complaints and disputes | Queue by age; evidence bundle; decision form; communicate with both parties | SUP, OPS |
| 16 | Exceptions queue | No-worker bookings, expiring KYC, price anomalies, stuck jobs, failed notifications, failed payouts | OPS |
| 17 | Analytics | Funnel, conversion, response rate, lead-to-job, average quote and job value, cancellation and no-show rates, repeat rate, worker retention, revenue per job, acquisition costs | All (scoped) |
| 18 | Fraud and flags | Duplicate-account signals, price anomalies, leakage signals, repeated cancellations; set action | OPS, SA |
| 19 | Templates | Notification templates and languages | OPS |
| 20 | Audit log | Search who, what, when, before and after, reason | SA |
| 21 | Admin users | Create admins, assign roles, reset MFA, review access | SA |

## Z. Recommended folder structure

One monorepo so API contracts, event schemas and error codes are shared by every app and a change cannot silently break a client.

```
fixo/
├─ apps/
|  ├─ api/                      # NestJS modular monolith
|  |  ├─ src/
|  |  |  ├─ modules/
|  |  |  |  ├─ auth/ users/ geo/ catalogue/ workers/ verification/
|  |  |  |  ├─ bookings/ matching/ leads/ contact/ quotes/ jobs/
|  |  |  |  ├─ pricing/ revenue/ payments/ ledger/ payouts/
|  |  |  |  ├─ ratings/ trust/ notifications/ realtime/ media/
|  |  |  |  └─ config/ analytics/ audit/
|  |  |  ├─ platform/            # idempotency, outbox, queue, scheduler,
rate limit
|  |  |  ├─ common/              # guards, pipes, filters, error codes
|  |  |  └─ main.ts
|  |  ├─ db/
|  |  |  ├─ migrations/
|  |  |  └─ seeds/               # real reference data only: categories,
Lucknow areas
|  |  └─ test/                   # integration and end-to-end tests
|  ├─ jobs/                      # background worker entry point (same code,
own process)
|  ├─ customer-web/              # Next.js customer app (PWA)
|  ├─ admin-web/                 # Next.js admin panel
|  └─ worker-app/                # React Native (Expo) worker app
├─ packages/
|  ├─ contracts/                 # OpenAPI types, request schemas, event
schemas, error codes
|  ├─ ui/                        # shared components and design tokens
|  ├─ i18n/                      # Hindi and English strings
|  └─ config/                    # eslint, tsconfig, prettier
├─ infra/                        # containers, infrastructure as code, CI
pipelines
└─ docs/                         # this specification, decision records,
runbooks
```

### Inside every backend module

```
bookings/
├─ bookings.controller.ts        # routes, auth guards, request validation
├─ bookings.service.ts           # use cases, transactions
├─ domain/
│  ├─ booking.state-machine.ts   # allowed transitions, one place only
│  └─ booking.rules.ts           # pure business rules, easy to unit test
├─ bookings.repository.ts        # the only file that touches this module's
tables
├─ dto/                          # request and response shapes
├─ events/                       # event definitions published to the outbox
└─ bookings.spec.ts              # unit and integration tests
```

The state machine files are plain code with no framework dependency, so every transition in sections E–J has a unit test that lists the allowed and forbidden moves.

## Open business decisions

These rules are ambiguous in the brief or depend on your business strategy. The system is built so each one is configuration, but a default must be chosen for launch. Please confirm or change each recommendation.

| ID | Decision | Options | Recommendation |
|---|---|---|---|
| OBD-1 | Primary revenue model | A pay per lead; B prepaid credits; C subscription; D commission; E hybrid | Launch with B (small lead fee from prepaid credits) and add commission on online-paid jobs once in-app payment adoption is healthy |
| OBD-2 | When a lead fee is charged, and refunds | Charge at Interested tap; at contact access; only when customer shortlists. Refund if customer unreachable, booking fake or out of area, or lead filled | Charge at contact access only; auto-refund for filled leads (never charged), unreachable customers after set attempts, and fake or out-of-area bookings |

| ID | Decision | Options | Recommendation |
|---|---|---|---|
| OBD-3 | Cap and selection | Cap of 1, 3 or 5; customer chooses among quotes; FIXO assigns the best; first-come | Cap of 3 (your example); customer chooses among quotes; optional "pick for me" for ASAP |
| OBD-4 | Must the customer approve every quote in-app? | Always; only above an amount; never (verbal) | Always in V1: it creates the written record that disputes and price control depend on |
| OBD-5 | Collecting FIXO's share on cash jobs | Ignore (lead fee only); deduct from credits; weekly dues; block leads above a dues limit | Lead fee is the primary revenue at launch; if commission is added, record dues in the ledger and block new leads above a limit |
| OBD-6 | What a worker sees and when | Area only until access; full address after quote acceptance; direct vs masked number | Area, landmark and distance at the lead; masked call after access; exact address and customer name after the quote is accepted |
| OBD-7 | Fee ownership and tax | Who bears gateway fees; whether to charge a customer booking fee; GST and invoice rules | Confirm with an accountant. Suggested defaults: FIXO absorbs gateway fees, no customer booking fee at launch (conversion matters more) |
| OBD-8 | Cancellation windows, fees and penalty points | Free-cancel window length; fees at each stage; penalty weights | Start light: no customer fee before CONFIRMED; modest worker penalty points; review after the first month of data |
| OBD-9 | Payment timing | Pay after job; partial advance for material; hold until confirmation | Pay after job in V1 (online or cash); add advances for large material jobs in V2 |

| ID | Decision | Options | Recommendation |
|---|---|---|---|
| OBD-10 | Visibility of worker ratings of customers | Private to FIXO; visible to workers as a score; visible as comments | Private to FIXO, used for restrictions and safety |
| OBD-11 | Customer app form | PWA or web first; native app from day one | Web/PWA first; a native customer app after launch metrics; native worker app from day one because push alerts matter |
| OBD-12 | Workers with identity verified but skill not yet verified | Strict (no leads); probation with limited leads and monitoring | Probation: limited leads in one category, trial-job feedback, then full skill verification |
| OBD-13 | KYC scope and legal handling | Which documents per category; verification provider; how identifiers are stored | Use a licensed verification provider; store results and masked references; legal review before launch |
| OBD-14 | Service guarantee | Days of rework cover; who pays; what is covered | Simple rework guarantee for on-platform jobs, worker bears rework, FIXO mediates; no FIXO-funded warranty in the MVP |
| OBD-15 | Emergency service | Hours offered; fee level and split between worker and FIXO; radius | Limited hours at launch; separate emergency fee with a share to the worker; larger radius |

# Approval checklist and implementation phases

### What I need from you before Phase 1 starts

- [ ] Confirm or change the recommendation for each of OBD-1 to OBD-15.
- [ ] Approve the separate state machines (booking, lead, quote, job, payment) in place of the single status chain in your brief (section E).
- [ ] Approve the technology stack, or name your preferred alternatives (section A).
- [ ] Approve the MVP boundary in section U.

- [ ] Provide real reference data for seeding: the Lucknow areas you want active, the service categories and problems, and starting price ranges. Nothing is invented; anything missing stays empty and is marked as not configured.
- [ ] Choose providers, or ask me to compare them: payment gateway, SMS/OTP, WhatsApp, masked calling, KYC verification, maps.

**Implementation phases (after approval)**

Each phase ends with tests, an authorization review, a database consistency check, a duplicate-request check, a mobile responsiveness check where relevant, and a written report of what changed. No business rule changes silently between phases.

| Phase | Scope | Exit criteria |
|---|---|---|
| 1 | Monorepo, database, migrations, auth, roles, config registry, audit, idempotency, outbox | Login works for all three roles; RBAC tests pass; audit rows written |
| 2 | Worker onboarding, KYC and skill verification | Full onboarding to VERIFIED through the admin queue; private document access verified |
| 3 | Geography, catalogue, customer service discovery | Admin edits appear in the app without deployment |
| 4 | Booking and matching | Booking creates a ranked list with a stored match log; duplicate protection works |
| 5 | Lead system and controlled contact | Cap holds under concurrent load test; masked contact flow works |
| 6 | Quotes and job lifecycle | All transitions covered by state-machine tests; cancellation matrix implemented |
| 7 | Payments, ledger, revenue engine | Webhook idempotency proven; ledger always balances; worker earnings reconcile |
| 8 | Ratings and complaints | Dispute hold and resolution paths work end to end |

| Phase | Scope | Exit criteria |
|---|---|---|
| 9 | Admin panel completion | Every module in section Y usable with real data |
| 10 | Realtime and notifications | No duplicate subscriptions or notifications under reconnect tests |
| 11 | Security and performance hardening | Review checklist complete; load test at agreed targets; backup restore tested |
| 12 | Testing and production deployment | Staging sign-off; launch checklist; monitoring and runbooks in place |

I will not begin Phase 1 until you approve this specification.